

Mastercard Innovation Challenge 2026

AI Defence Lab for Payment Security — First-Cut Solution Architecture & Implementation Brief

Revision focus: The closed-loop feedback path has been corrected so that detector findings are routed through **Agent 3** → **Attack Memory** → **Agent 1**. Agent 1 becomes a better researcher over successive rounds; only its resulting attack hypothesis/research output is passed to Agent 2. Agent 2 does **not** receive direct detector feedback.

Purpose: build specification for the first implementation cut. This document consolidates the official challenge statement/rules from the provided HTML snapshots and the proposed architecture discussed by the team. Architecture/model choices are explicitly treated as proposed engineering design unless the supplied official material states otherwise.

Source basis: Provided Kaggle HTML snapshots (overview/rules) and the participant's architecture sketch.

1. Challenge at a Glance

The challenge asks teams to build one end-to-end Red Teaming AI system for payment security across **Identify, Generate, and Defend**. The central idea is a closed loop: attacks generated by the red team become training/stress-testing material for the defense, while gaps revealed by the defense feed back into new attack ideas.

Pillar	Official expectation	First-cut interpretation
Identify / Ideate	Research and map emerging, novel GenAI-powered payment-fraud attacks; be thorough and exhaustive across channels, rails and social-engineering surfaces.	Agent 1 researches using RAG + Attack/Experiment Memory. Detector findings enter memory and improve Agent 1's next research cycle.
Generate	Generate/simulate attacks at scale with high fidelity so synthetic attacks resemble realistic payment behaviour, distributions and edge cases.	Agent 2 converts Agent 1's hypothesis into a structured specification; a conditional tabular generator instantiates it.
Defend	Build an AI/ML solution that detects, flags or mitigates generated attacks; prioritize detection efficacy and low false positives on legitimate payments.	Blue-team detector; start with a strong tabular baseline and upgrade only if justified by results.

Important: the official challenge does not prescribe an LLM, RAG, GAN, diffusion model, CTGAN, a particular detector, or a specific multi-agent architecture. These are proposed engineering choices.

2. Official Rules & Submission Constraints

Area	Rule / implication
Eligibility	Individual or team of up to 5; each participant may belong to only one team; only one submission per team; registered team composition constraints apply; participants must be 18+; Mastercard employees/contractors/etc. are excluded.
Deadline	Registration: 10 Aug 2026–20 Aug 2026. Submission deadline: 31 Aug 2026. Draft/unsubmitted work after the deadline is not eligible.
Required artifacts	Complete runnable GitHub repository covering Identify/Generate/Defend; solution walkthrough; working web prototype with presentable UI.
Data	Use only synthetic, anonymized, or authorized sample data. Do not use real cardholder data, PII, or production payment data.
Adversarial testing	Keep testing within event scope. Do not target live systems, payment infrastructure, or third parties.
Integrity	No plagiarism, cheating, fraud, misrepresentation, unauthorized collaboration, or other conduct undermining event fairness.
Confidentiality	Do not include confidential/proprietary/trade-secret information from employers, customers, clients, partners or other third parties unless authorized.
IP	Participants retain IP ownership; Mastercard receives the stated non-exclusive, royalty-free, irrevocable, perpetual, worldwide license for event administration/judging/promotion/documentation.

LLM/AI usage: the supplied Terms do not expressly prohibit or prescribe LLM use. Therefore LLMs are treated here as an implementation choice, subject to originality, rights, confidentiality, data and cybersecurity constraints.

3. Proposed First-Cut Architecture

The architecture below is the corrected version of the team's proposed design. The key feedback-path rule is: **Agent 3 does not directly instruct Agent 2**. Agent 3 analyzes suitable evaluation evidence and detector behaviour, writes structured weaknesses and next-attack directions into **Attack / Experiment Memory**, and Agent 1 retrieves that accumulated feedback. Agent 1 therefore becomes a better researcher over successive rounds. Only the resulting Agent 1 research output/hypothesis is passed to Agent 2 for specification. Only the resulting Agent 1 research output/hypothesis is passed to Agent 2 for specification.

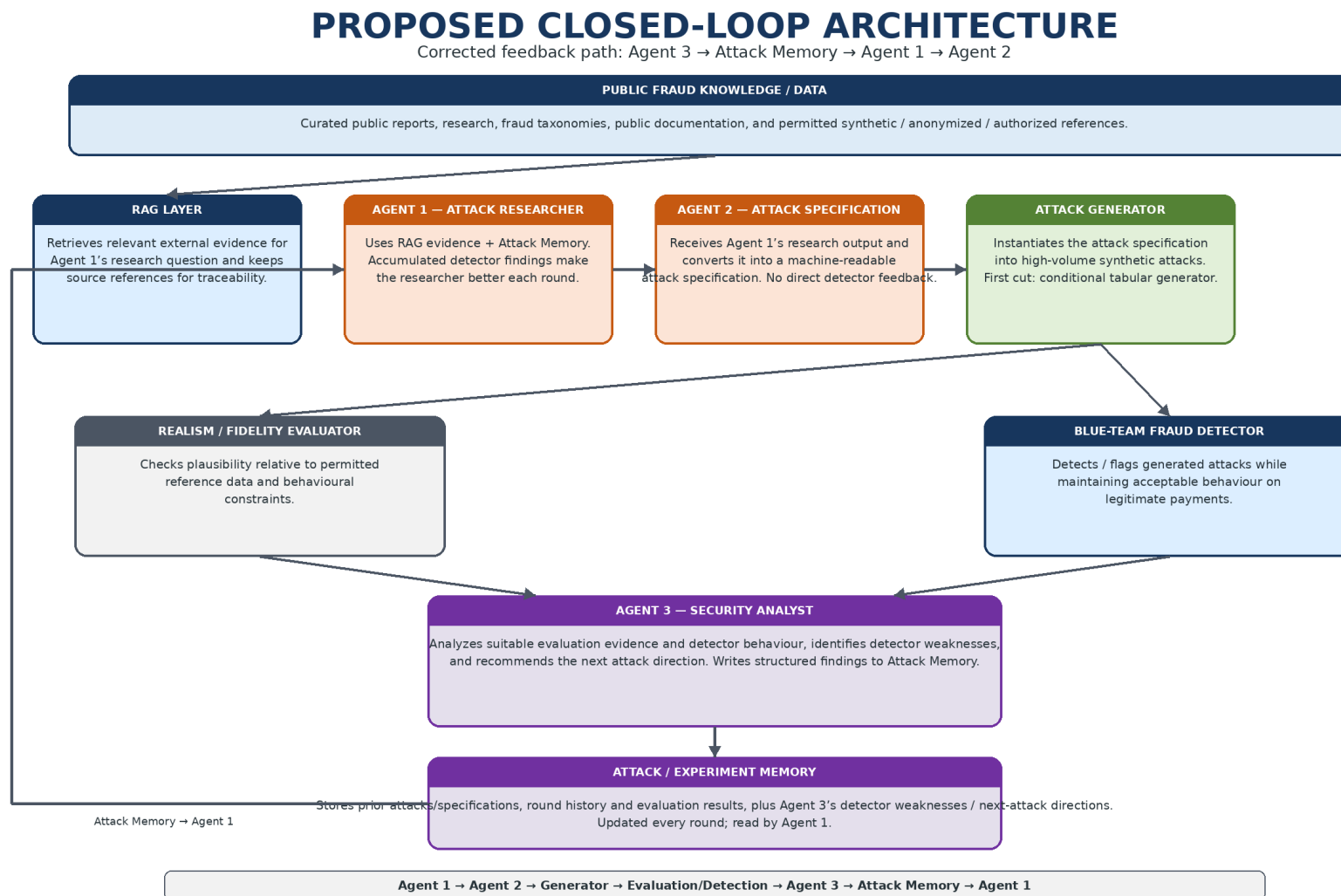


Figure 1. Corrected architecture. Attack Memory is the adaptive bridge: Agent 3 writes structured detector-weakness findings and next-attack directions to memory; Agent 1 retrieves them and becomes a better researcher. There is no direct Agent 3 → Agent 2 feedback path.

4. Component-by-Component Specification

4.1 Public Fraud Knowledge / Data

Bootstrap source for external grounding. It contains permitted public or authorized material such as fraud/security reports, research papers, payment-fraud taxonomies, public documentation and synthetic/authorized payment references. It answers: “What is already known outside our system?” It is not the adaptive memory of the experiment.

Implementation guidance: Implementation: curate documents, attach metadata, chunk them, and expose them to the RAG layer. Do not ingest confidential or unauthorized material.

4.2 RAG Layer

RAG (Retrieval-Augmented Generation) is the retrieval mechanism that selects relevant evidence from the public knowledge base and supplies it to Agent 1 before research reasoning. It is not a separate foundation model.

Implementation guidance: Input: Agent 1 research query. Output: top-k evidence snippets + source metadata. Keep evidence references attached to each hypothesis so the system can distinguish known evidence from proposed novelty.

4.3 Attack / Experiment Memory

Attack Memory is the system’s internal experience store. It is **not the same as the RAG knowledge base**. It records what the closed-loop system has already attempted and learned: prior attack hypotheses/specifications, generated-attack history, evaluation summaries, detector findings, discovered weaknesses and recommended next-attack directions. Its purpose is to prevent repetition and make Agent 1 progressively better informed.

Implementation guidance: **Update path:** after each round, store the relevant attack/specification and evaluation history; Agent 3 additionally writes structured detector weaknesses and next-attack directions. **Read path:** Agent 1 retrieves this memory in its next research cycle. This is the explicit adaptive feedback bridge. Recommended first cut: SQLite/JSON/Parquet with structured records; add vector retrieval later if semantic search becomes useful.

4.4 Agent 1 — Attack Researcher LLM

The adaptive research component. It identifies plausible, diverse and potentially novel GenAI-enabled payment-fraud attack hypotheses by combining **RAG evidence + Attack Memory + challenge scope**. Crucially, it also receives accumulated detector feedback through Attack Memory. Therefore, after every round, its research context becomes richer: it can avoid previously explored ideas and deliberately investigate directions suggested by observed detector weaknesses.

Implementation guidance: Input: retrieved public evidence + relevant Attack Memory + safety/scope constraints. Output: structured **AttackHypothesis** with attack family, scenario, target context, behavioural mechanism, novelty rationale, evidence references and research direction. It should not directly generate raw transactions. Optional controlled periodic external search can refresh the knowledge base; continuous unrestricted browsing is not required.

4.5 Agent 2 — Attack Specification Strategist LLM

Converts Agent 1’s research output into a machine-readable simulation recipe. Agent 2 is downstream of the researcher; it does **not** receive direct detector feedback. This separation is intentional: Agent 1 performs adaptive research, while Agent 2 operationalizes the selected research direction.

Implementation guidance: Input: **Agent 1 AttackHypothesis** + generation/schema constraints. Output: **AttackSpecification** describing scenario, temporal/amount/device/beneficiary behaviour, feature constraints, realism constraints, difficulty/evasion objective and evidence references. The specification should be precise enough for the generator to execute without another LLM step.

4.6 Attack Generator

Numerical generation layer that instantiates the specification into high-volume synthetic attack transactions. It is not an LLM conversation.

Implementation guidance: First cut: conditional tabular generator. CTGAN-style generation is a practical baseline; tabular diffusion or an adversarial generator can be tested later. Condition generation on the attack specification and preserve realistic feature relationships. Every generated record should retain attack ID, generation metadata and ground-truth attack label.

4.7 Realism / Fidelity Evaluator

Independent evaluator answering: “Are these generated attacks plausible payment behaviours and faithful to the specified scenario?” It should remain separate from the fraud detector because realism and detectability are different objectives.

Implementation guidance: Use permitted reference/synthetic data to compare distributions, dependencies and behavioural constraints. Produce a structured fidelity report that can be stored in memory. The exact composite score is a proposed internal evaluation design, not an official Mastercard formula.

4.8 Blue-Team Fraud Detector

The defensive ML model. It evaluates the generated attacks and legitimate holdout data and produces fraud risk/labels. Its job is to maximize useful detection while maintaining acceptable behaviour on legitimate payments.

Implementation guidance: First cut: LightGBM/XGBoost on engineered tabular features. Upgrade to sequence/graph models only if the data structure and experiments justify it. Keep an unseen holdout so the detector is not merely evaluated on samples it already trained on.

4.9 Agent 3 — Security Analyst LLM

The post-detection reasoning component. It analyzes suitable evaluation evidence and detector behaviour, identifies **detector weaknesses**, and recommends a **next attack direction**. It should not be hard-coded to one particular error type or single metric. The output is a structured analysis that can be compared across rounds.

Implementation guidance: Input: detector outputs and appropriate evaluation evidence + attack/specification metadata + fidelity/evaluation summaries + historical context as needed. Output: **WeaknessReport** containing observed weakness patterns, evidence supporting the finding, confidence/priority where useful, and a recommended next attack direction. Its feedback is written to **Attack Memory only**; Agent 1 retrieves it in the next research cycle.

4.10 Closed-Loop Controller

Orchestrates autonomous rounds and enforces the correct information flow. The controller should make the feedback path explicit in code: Agent 3 → Attack Memory → Agent 1 → Agent 2.

Implementation guidance: Round: Research → Specify → Generate → Fidelity/Evaluation → Detect → Analyze → Memory update → next research cycle. Human control should define permitted data, safety boundaries, budgets and stopping rules; the loop itself can then run without manual attack selection.

5. Exact Data Contracts Between Blocks

Stable interfaces let two teammates develop in parallel. The most important correction is that Agent 3's output is written to Attack Memory and consumed by Agent 1; Agent 2 receives Agent 1's resulting research hypothesis, not direct detector feedback.

Interface	Input	Output
Agent 1	RAG evidence + relevant Attack Memory + scope/safety constraints	AttackHypothesis JSON
Agent 2	Agent 1 AttackHypothesis + generation/schema constraints	AttackSpecification JSON
Generator	AttackSpecification + permitted reference/synthetic schema	SyntheticTransactions + attack labels + generation metadata
Realism evaluator	Synthetic transactions + permitted reference data	FidelityReport
Detector	Training data for fitting; transactions for inference	Risk score / predicted label / confidence
Agent 3	Detector/evaluation evidence + attack metadata + fidelity summary + history	WeaknessReport + next-attack direction
Attack Memory	Attack/specification history + evaluation results + Agent 3 WeaknessReport	Retrievable experience context for Agent 1

Recommended minimal AttackSpecification fields: attack_id, attack_family, scenario, target_context, transaction_pattern, temporal_pattern, amount_pattern, device_pattern, beneficiary_pattern, required_features, realism_constraints, evasion_goal, evidence_refs.

Recommended minimal WeaknessReport fields: round_id, observed_pattern, affected_attack_context, supporting_evidence, weakness_summary, priority/confidence, recommended_next_attack_direction, memory_refs.

6. Closed-Loop Operation — Corrected Example

Round 0 — Bootstrap

Start with the curated public knowledge base, permitted reference/synthetic transaction schema, empty/seeded Attack Memory, and a baseline detector.

1 — Identify

Agent 1 queries RAG and retrieves relevant Attack Memory. It proposes an attack hypothesis. In later rounds, the memory includes Agent 3's detector-weakness findings, so Agent 1's research is explicitly informed by what the blue team has already exposed.

2 — Specify

Agent 2 receives the selected Agent 1 hypothesis and converts it into a machine-readable AttackSpecification. There is no direct Agent 3 → Agent 2 feedback path.

3 — Generate

The generator creates synthetic attacks according to the specification.

4 — Evaluate realism

The realism/fidelity evaluator checks whether the generated samples remain plausible and consistent with the specified behaviour.

5 — Defend

The blue-team detector evaluates generated attacks and an appropriate legitimate holdout. Record the relevant evaluation evidence for the round.

6 — Analyze

Agent 3 analyzes suitable metrics/evidence and detector behaviour, identifies weaknesses, and recommends a next attack direction. Do not hard-code the reasoning to one particular metric or error type.

7 — Remember

Write the attack history, evaluation results and Agent 3's structured weakness report into Attack Memory.

8 — Research again

Agent 1 retrieves the updated Attack Memory together with external evidence. It now has more information about what has already been tried and where the detector is weak, so it proposes a better-informed next research direction.

9 — Repeat

The resulting Agent 1 hypothesis again goes to Agent 2 → Generator → Evaluation/Detection → Agent 3 → Attack Memory → Agent 1.

7. Why Not Just a Standard GAN?

A GAN discriminator that only distinguishes real from synthetic samples is not automatically the same thing as the blue-team fraud detector. The challenge needs both realistic synthetic attacks and effective fraud detection. Therefore keep **fidelity evaluation** and **fraud detection** as separate objectives in the first cut.

An advanced experiment can later formulate a red-team/blue-team objective in which the generator is rewarded for attacks that are simultaneously realistic, sufficiently novel and difficult for the current detector, while the detector improves against those attacks. The important architectural point remains: Agent 3 analyzes the result, stores the weakness in Attack Memory, and Agent 1 uses that experience to improve the next research cycle.

Objective	Question	Primary owner
Fidelity	Do generated attacks resemble plausible payment behaviour?	Realism evaluator
Evasion difficulty	Does the attack expose a weakness in the current detector?	Generator + Agent 3 analysis

Objective	Question	Primary owner
Detection	Can the blue team correctly identify the attack?	Fraud detector
Diversity / novelty	Is the next attack meaningfully different from prior explored attacks?	Agent 1 + Attack Memory + evaluation

8. Official Evaluation Criteria & Recommended Internal Metrics

The supplied challenge material explicitly evaluates: **diversity of attacks identified, fidelity of attacks in simulation, detection algorithm efficacy, novelty of the solution, and real-world feasibility in live payments**. For detection, the overview explicitly calls out precision, recall, F1/AUC and low false positives. The official material does not supply a numerical weighting or a detailed formula for the other dimensions; therefore the additional metrics below are recommended internal evaluation methodology, not official Mastercard scoring formulas.

8.1 Detection efficacy

Precision, recall, F1, ROC-AUC and false-positive behaviour should be reported as the core detector evidence. Supplement with PR-AUC where class imbalance makes it informative, confusion matrices for auditability, and calibration analysis if the detector exposes probabilities. Report overall and, where meaningful, by attack family/round.

8.2 Attack diversity

Define an explicit attack taxonomy. Report family coverage, scenario/mechanism diversity, behavioural/feature diversity, round-wise novelty of specifications, and deduplication checks so paraphrases are not counted as distinct attacks.

8.3 Attack fidelity

Evaluate marginal-distribution similarity, dependence/correlation similarity, conditional relationships, behavioural plausibility, downstream utility of synthetic data, and privacy/leakage sanity checks. Keep the components separately visible before forming any composite score.

8.4 Novelty

Measure distance from prior Attack Memory in structured/semantic space; identify whether new attacks reveal previously unseen detector weaknesses; distinguish mechanism novelty from simple parameter changes; retain evidence references for the researcher's novelty claims.

8.5 Real-world feasibility

Report inference latency, throughput, resource requirements, deployment shape, robustness under distribution shift, and auditability/provenance. Emphasize that red-team generation is an offline stress-testing process and should not be placed in a live-payment critical path.

Reviewer-level principle: *do not optimize one metric in isolation. A high-diversity attack set that is unrealistic is not useful; a highly realistic generator that produces only near-duplicates is weak; and a detector with excellent recall but unacceptable false-positive behaviour is not deployment-ready. Report the trade-offs and show how the closed loop improves robustness across successive, increasingly informed attack sets.*

9. Experimental Protocol — Recommended

Use a clean separation between reference data, generator fitting data, detector training data and unseen evaluation data. A closed-loop result is credible only if later attacks are not simply copies of samples the detector has already seen.

Stage	Recommended protocol
Reference set	Permitted synthetic/anonymized/authorized data establishing legitimate/fraud distributions and feature relationships.
Generator training	Fit the generator without access to the final detector evaluation set.
Red-team generation	Generate multiple attack families/variants with stable attack IDs and generation metadata.
Fidelity evaluation	Run realism checks before using generated samples as evidence of detector performance.
Detector training	Document exactly which generated attacks are included in training and keep a clean evaluation set.
Unseen attack test	Hold out attack variants/families or later-round attacks to measure generalization rather than memorization.
Closed-loop comparison	Compare detector performance before and after incorporating feedback-generated attacks.
Ablations	At minimum: baseline detector; static synthetic attacks; adaptive feedback loop; and, if feasible, alternative generator or feedback strategy.

10. First-Cut Model Strategy

Layer	First cut	Upgrade only if useful
LLM agents	One shared open-source/hosted LLM used in three logical roles: Researcher, Strategist, Security Analyst.	Different LLMs per role only as an ablation; separate models add compute/deployment complexity.
RAG	Embedding retrieval over curated public knowledge.	Hybrid retrieval/reranking/controlled periodic search.
Memory	SQLite/JSON/Parquet structured experiment store.	Structured + vector retrieval if semantic memory search is needed.
Generator	Conditional tabular generator baseline.	Tabular diffusion or explicit adversarial generator with realism + difficulty objectives.
Detector	LightGBM/XGBoost baseline.	Sequence Transformer/GNN if structure materially improves results.
Feedback	Agent 3 → Attack Memory → Agent 1; Agent 1 hypothesis → Agent 2.	Learned attack-policy prioritization if time permits.
UI	Single-page loop demo with attack selection, generation, evaluation, detection and feedback cards.	Streaming visualizations and richer round history.

13. Human vs. Autonomous Responsibilities

Human-controlled	Autonomous after initialization
Choose/approve source materials and permitted datasets.	Retrieve relevant public evidence for the current research question.
Define safety boundaries, schemas, compute budgets and stopping rules.	Retrieve Attack Memory and incorporate prior detector findings into research.
Review generated attack families and final results.	Generate attack hypotheses, convert hypotheses to specifications, generate attacks, evaluate fidelity, run detector, analyze results, update memory and start the next research cycle.
Decide final submission claims and validate reproducibility.	Repeat the loop within the configured safety/data/compute boundaries.

Safety boundary: *autonomous means autonomous experimentation inside pre-defined scope, data, safety and compute constraints. It does not authorize attacks against live payment systems, infrastructure or third parties.*

14. What GitHub Copilot Should Build First

- Create the project skeleton and configuration system.
- Define AttackHypothesis, AttackSpecification, GeneratedTransaction, DetectionResult, FidelityReport, WeaknessReport and MemoryRecord schemas.
- Implement a small RAG pipeline over a curated local document set.
- Implement Agent 1, Agent 2 and Agent 3 using one shared LLM with distinct role prompts. Make the Agent 3 → Attack Memory → Agent 1 path explicit in code.
- Implement a deterministic/simple conditional synthetic generator baseline so the loop works before GPU-heavy training.
- Implement a LightGBM/XGBoost detector and the core detection evaluation suite.
- Implement the realism/fidelity evaluator and persist its report.
- Implement Attack Memory and the controller so multiple rounds execute without manual attack selection.
- Only after the loop is stable, replace the baseline generator with CTGAN/tabular diffusion/adversarial generation and test whether the evaluation evidence improves.
- Build the web prototype around the actual information flow: research hypothesis → specification → generated attacks → fidelity → detection → Agent 3 analysis → Attack Memory → next Agent 1 research cycle.

15. Final Acceptance Checklist

- All three official pillars are present: Identify, Generate, Defend.
- Agent 1 has access to both RAG evidence and accumulated Attack Memory.
- Agent 3 feedback is written to Attack Memory and is consumed by Agent 1; there is no direct Agent 3 → Agent 2 feedback path.
- Agent 2 receives the Agent 1 research hypothesis and converts it into a specification.
- At least two closed-loop rounds can execute with no manual attack selection after initialization.
- The second round demonstrably uses accumulated detector findings to change the research direction.
- Attack Memory records prior hypotheses/specifications/results and prevents trivial repetition.
- Fidelity is evaluated before generated attacks are used as evidence of detector performance.
- Detection is evaluated on an appropriate unseen/held-out set.
- Officially emphasized detection measures and false-positive behaviour are reported; additional internal metrics are explicitly defined.
- Attack diversity, fidelity, novelty and real-world feasibility are documented with transparent internal definitions.
- GitHub repository is runnable, documented and reproducible.
- Web prototype visibly demonstrates the closed loop and the feedback path.
- No real cardholder/PII/production payment data is used and no live systems/third parties are targeted.
- All final claims are supported by actual experiments and no unauthorized confidential material is included.

16. Source Fidelity & Interpretation Notes

Officially stated in the supplied HTML: the challenge requires Identify/Generate/Defend; emphasizes diversity of attacks, fidelity of attacks, detector efficacy, novelty and real-world feasibility; explicitly mentions precision/recall/F1/AUC and low false positives; requires the repository, walkthrough and web prototype; and imposes data, adversarial-testing, integrity, confidentiality and IP constraints.

Not officially prescribed in the supplied HTML: RAG, three LLM agents, a shared LLM, CTGAN, diffusion, GAN/minimax training, LightGBM, Transformers, GNNs, a particular attack taxonomy, numerical judging weights, or a specific fidelity/diversity/novelty formula. These remain proposed engineering choices.

Most important design principle: the system should demonstrate a genuine adaptive loop. The strongest demo is not “LLM generates fraud and classifier detects it”; it is “researcher uses external evidence + accumulated experiment memory → attack hypothesis → specification → high-fidelity synthetic attacks → detector stress test → Agent 3 analyzes suitable evidence and identifies weakness → weakness is stored in Attack Memory → Agent 1 uses that feedback to become a better researcher → next hypothesis.”